

Instructions for Lab Submission

1. File Naming and Submission:

- Submit your solution as a **ZIP file** named `lab<number>.<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
- Ensure that the ZIP file contains:
 - C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
 - No unnecessary files (such as executables, temporary files, or editor configs).
- Do not submit **TEXT(.txt)** or **EXE(.exe)** files.

2. Code Organization:

- Write each task in its own `.cpp` file (as specified above).
- Ensure that your code is modular and logically separated into functions when specified.

3. Code Neatness:

- Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
- Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
- Add clear explanations where required (or specified)** to make the logic of your program easy to follow.

4. Code Compilation and Functionality:

- Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
- Double-check that the functionality of the program meets the requirements outlined in the task.

5. Additional Instructions:

- Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
- Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
- If you encounter any issues or require clarifications, reach out before the submission deadline.

6. Deadline:

- Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
- Double-check that the zip file is correctly named and contains all necessary files before submitting.

7. Plagiarism Policy:

- The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

The for Loop

A **for** loop is used when a block of code needs to be repeated a known number of times.

The general form is:

```
for (initialization; condition; update) {  
    // statements  
}
```

The three parts are:

- **Initialization:** sets the starting value.
- **Condition:** determines whether the loop continues.
- **Update:** changes the loop variable after each iteration.

Example:

```
for (int i = 1; i <= 4; i++) {  
    cout << i << endl;  
}
```

Output:

```
1  
2  
3  
4
```

Counters and Accumulators

A **counter** keeps track of how many times something occurs.

```
int count = 0;  
  
for (int i = 1; i <= 5; i++) {  
    if (i % 2 == 0) {  
        count++;  
    }  
}  
  
cout << count;
```

An **accumulator** keeps a running total.

```
int total = 0;  
  
for (int i = 1; i <= 4; i++) {  
    total = total + i;  
}  
  
cout << total;
```

The initial value is important. A sum normally starts from **0**, while a product normally starts from **1**.

Using Conditions Inside a Loop

An **if** statement can be placed inside a loop when a decision must be made during each iteration.

```
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) {
        cout << i << " is even" << endl;
    }
    else {
        cout << i << " is odd" << endl;
    }
}
```

The condition is checked separately during every iteration.

The break Statement in Loops

The `break` statement immediately terminates the loop.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    cout << i << " ";
}
```

Output:

```
1 2 3 4
```

Once `break` executes, control moves to the first statement after the loop.

The continue Statement

The `continue` statement skips the remaining statements of the current iteration and moves to the next iteration.

```
for (int i = 1; i <= 6; i++) {
    if (i == 3) {
        continue;
    }
    cout << i << " ";
}
```

Output:

```
1 2 4 5 6
```

The loop itself does not end. Only the current iteration is skipped.

break vs. continue

The two statements behave differently:

Statement	Effect
<code>break</code>	Terminates the entire loop
<code>continue</code>	Skips only the current iteration

Boolean Variables

The `bool` data type stores one of two values:

- `true`
- `false`

Boolean variables are useful for remembering whether an event has occurred.

```
bool found = false;

for (int i = 1; i <= 5; i++) {
    if (i == 4) {
        found = true;
    }
}

if (found == true) {
    cout << "Value found";
}
```

Such variables are sometimes called **flags**.

Random Numbers Inside a Loop

A new random value can be generated during every iteration of a loop.

Remember to seed the random number generator only once:

```
srand(time(0));

for (int i = 1; i <= 5; i++) {
    int number = rand() % 10 + 1;

    cout << number << endl;
}
```

Here, each iteration generates a number from `1` to `10`.

The general formula for generating a value from `1` to `N` is:

```
rand() % N + 1
```

Common C++ Errors

Small syntax or logical mistakes can cause a program to behave incorrectly.

1. Missing Semicolon

Incorrect:

```
cout << "Hello"
```

Correct:

```
cout << "Hello";
```

2. Assignment Instead of Comparison

Incorrect:

```
if (x = 5)
```

Correct:

```
if (x == 5)
```

The operator `=` assigns a value, while `==` compares two values.

3. Accidental Semicolon After a Loop

Incorrect:

```
for (int i = 0; i < 5; i++); {  
    cout << "Hello";  
}
```

Correct:

```
for (int i = 0; i < 5; i++) {  
    cout << "Hello";  
}
```

The semicolon after the loop condition causes the loop body to become separate from the loop.

4. Incorrect Assignment to a Boolean Variable

Incorrect:

```
valid == false;
```

Correct:

```
valid = false;
```

Tracing a Loop

Tracing means following the values of variables one iteration at a time.

Consider:

```
int total = 0;  
  
for (int i = 1; i <= 3; i++) {  
    total = total + i;  
}
```

The values change as follows:

i	total after iteration
1	1
2	3
3	6

Tracing a program is useful for finding logical errors and understanding how a loop changes variables during execution.

Lab Tasks

Task 1: Gambler's Fortune

A gambler enters a game with a fixed amount of money and decides to play a fixed number of rounds. In each round, the gambler either wins or loses one unit of money.

Problem Description

The gambler begins with some starting money and plays exactly N rounds. In each round, the program randomly generates either 0 or 1.

The complete algorithm is given below:

```

START

INPUT starting_money
INPUT N

SET money = starting_money

REPEAT N TIMES

    GENERATE a random number: 0 or 1

    IF random number is 1
        INCREASE money by 1
    ELSE
        DECREASE money by 1
    END IF

END REPEAT

DISPLAY final money

IF final money > starting_money
    DISPLAY "Profit"
ELSE IF final money < starting_money
    DISPLAY "Loss"
ELSE
    DISPLAY "No Change"
END IF

STOP
  
```

Convert the given algorithm into a working C++ program without changing its logic.

Sample Output

```

Enter starting money: 10
Enter number of rounds: 6

Final Money: 12
Result: Profit
  
```

Note: The final amount and result may be different between executions because the program uses random values.

Task 2: The Broken Prime Number Checker

A programmer has written a small program to determine whether a number is prime. Unfortunately, the program contains several errors and does not work as intended.

Problem Description

The following C++ program is intended to take an integer from the user and determine whether it is a prime number. However, several mistakes were made while writing the program.

```
#include <iostream>
using namespace std;

int main() {

    int number;
    bool isPrime = false;

    cout << "Enter a number: "
    cin >> number;

    if (number <= 1) {
        isPrime = false;
    }

    for (int i = 2; i < number; i--); {
        if (number % i = 0) {
            isPrime == false;
            break;
        }
    }

    if (isPrime = true) {
        cout << number << " is a prime number." << endl;
    } else {
        cout << number << " is not a prime number." << endl;
    }

    return 0;
}
```

Find and fix the errors so that the program correctly determines whether the entered number is prime.

Requirements

- Fix all syntax and logical errors in the provided program.
- Keep the overall structure and purpose of the provided code.
- Use the existing `for` loop to check possible divisors.
- Use the `isPrime` variable to store the result.
- The program must correctly handle numbers less than or equal to `1`.
- Do not completely rewrite the program with a different approach.

Test Cases

```
Input:
Enter a number: 13

Output:
13 is a prime number.
```

```
Input:
Enter a number: 12

Output:
12 is not a prime number.
```

Task 3: The Factorial Calculator

A mathematics student needs a small program to calculate the factorial of a number without performing the multiplication manually.

Problem Description

Write a C++ program that asks the user to enter a positive integer N .

The factorial of a number is the product of all positive integers from **1** up to that number.

For example:

$$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$$

Use a **for** loop to calculate the factorial of the entered number and display the final result.

Test Case

```

Input:
Enter a number: 5

Output:
Factorial of 5: 120
    
```

Output

Task 4: The Bus Boarding System

A university shuttle has a limited number of seats, and students are waiting to board before the morning lecture.

The program first asks how many students are waiting, the maximum number of seats available, and the maximum total number of bags allowed. Each student then enters the number of bags they are carrying.

A student carrying more than **2 bags** is considered a heavy load. For each student, display the student number, number of bags, total bags so far, and number of students currently boarded.

The program should process the students one by one and stop accepting further information as soon as the bus reaches its seat limit or bag limit, or all students have been processed.

At the end, display the total students boarded, total bags, number of heavy-load students, and whether the process was completed normally, stopped because of the seat limit, or stopped because of the bag limit.

Task 5: The Number Filter

A mathematics instructor gives students a sequence of numbers and asks them to investigate only the values that satisfy certain rules.

The program first takes two numbers, representing the beginning and ending points of the investigation. It then examines every number in this range.

For every number, display the number and the action taken. Numbers divisible by **3** should be ignored completely, while numbers divisible by **7** should immediately stop the investigation.

For every remaining number, display whether it is even or odd, add it to a running total, and increase the count of accepted values.

After each accepted number, display the current total and count.

At the end, display the final total of accepted numbers and how many numbers were included in the total.

Task 6: The Number Race

Two players, A and B, start with scores of 0. The program asks for the number of rounds and generates one random number between **1 and 20** in each round.

For every round, display the round number, generated number, and both player's scores before processing the number.

If the number is even, add it to A's score; if it is odd, add it to B's score. If the number is divisible by **5**, subtract 2 from the score of the player who received that number.

After processing each number, display both updated scores.

If either player's score becomes exactly **20**, that player wins immediately and no further rounds are performed.

If either score becomes greater than **20**, reset that player's score to 0 and display that the score was reset.

At the end, display both final scores and whether A won, B won, or the rounds ended without a winner.